# IT5007 Final Project Report

**Course:** IT5007 Software Engineering on Application Architecture
**Project:** CampusCart — NUS Campus Marketplace
**Group:** 14

Group number on Canvas: 14

| Student ID (AxxxxxxxZ) | NUSNet ID (exxxxxxx) | Name (as it appears on Canvas) |
|---|---|---|
| A0183438X | e0310233 | Ang Lee Chuan |
| A0183378R | e0310173 | Chou Han Xian, Aaron |
| A0329448R | e1553775 | Liaw Jian Wei |

---

## 1. Problem Statement & Motivation

The National University of Singapore has over 40,000 students, many of whom live on campus and cycle through textbooks, electronics, furniture, and other essentials every semester. Despite this constant demand, there is no dedicated, trusted channel for peer-to-peer student commerce. The existing alternatives present significant friction:

- **Lack of Trust:** On generic platforms like Carousell or Telegram, buyers have no way to verify whether a seller is a genuine NUS student. This creates safety and reliability concerns, particularly for high-value items or in-person meetups on campus.
- **Poor Discovery:** Telegram buy/sell groups are linear chat streams. Listings are buried within minutes, and there is no structured search, filtering, or categorization — making it nearly impossible for buyers to find specific items efficiently.
- **Inefficient Coordination:** The pervasive "PM me" culture across these channels wastes significant time on repetitive negotiations around pricing, item condition, availability, and meetup logistics.

**CampusCart** was conceived to directly address these pain points by providing a centralized, student-exclusive marketplace web application purpose-built for the NUS community. By requiring NUS email verification for access, structuring listings with categories and metadata, and offering direct offer mechanisms, CampusCart transforms the fragmented student trading experience into a trusted, searchable, and efficient digital marketplace.

---

## 2. Design & Prototyping

Before writing any code, the team invested in a thorough design phase to define the platform's user experience and information architecture. Using **Figma**, we created high-fidelity wireframes for every core screen of the application. This design-first approach served several critical purposes:

1. **Alignment:** The prototypes acted as a shared visual contract among all three team members, ensuring everyone had an identical understanding of the target product before diverging into parallel development streams.
2. **Scope Definition:** By explicitly designing each page, we could objectively assess the MVP scope and identify which features were essential versus aspirational, preventing scope creep during implementation.

3. **Architecture Guidance:** The component structure visible in the prototypes directly informed our React component hierarchy and GraphQL schema design, reducing rework during development.

## 2.1 Authentication & Entry Point

The login screen establishes CampusCart's identity as an NUS-exclusive platform from the very first interaction. The prominent "Sign in with NUS Email" call-to-action and the "Exclusively for the NUS Community" badge immediately communicate trust and exclusivity.

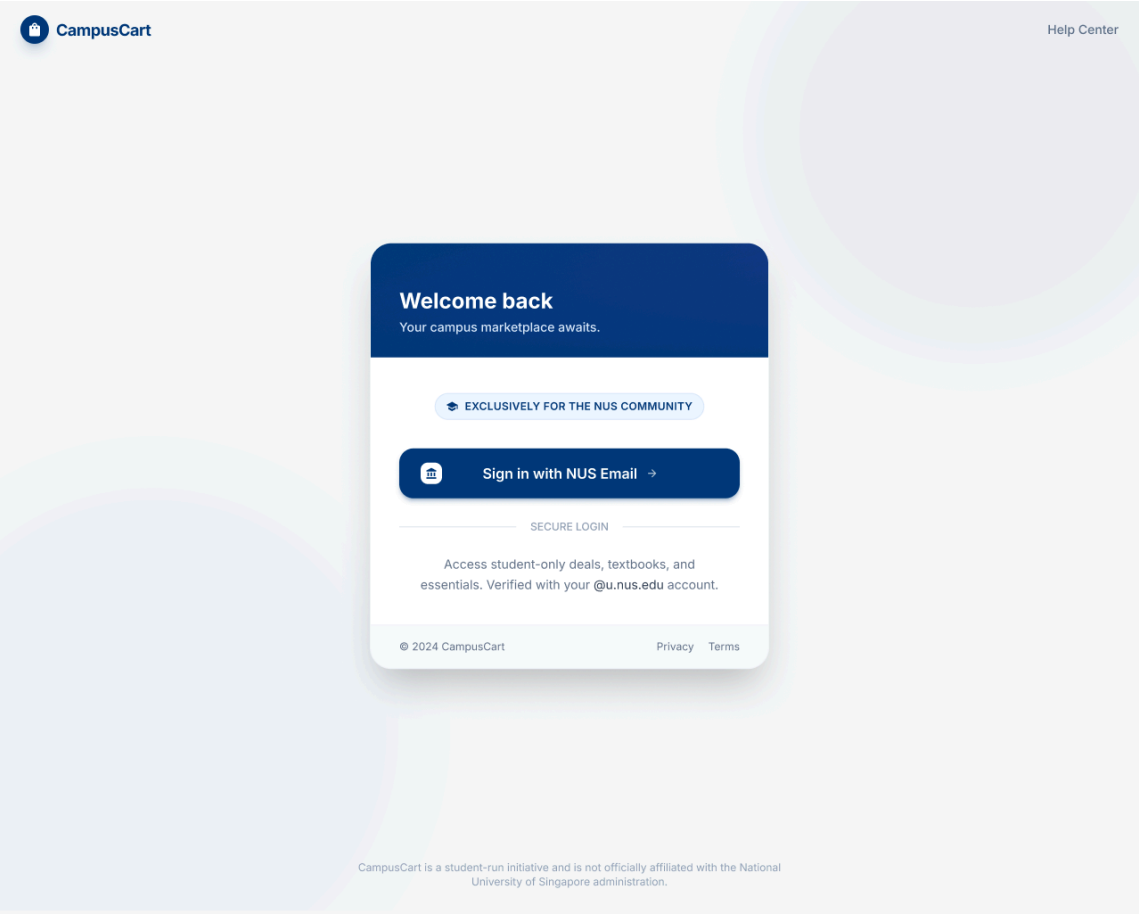



Figure 1: Login page prototype — emphasizing NUS-exclusive access and verified student identity.

## 2.2 Marketplace Browse & Discovery

The homepage serves as the primary discovery surface. The prototype establishes the core browsing experience: a hero search bar, category quick-filters (Textbooks, Furniture, Electronics, Clothing), and a card-based listing grid with condition badges, pricing, location tags, and seller verification status. A sidebar provides advanced filtering by condition, price range, and campus location.

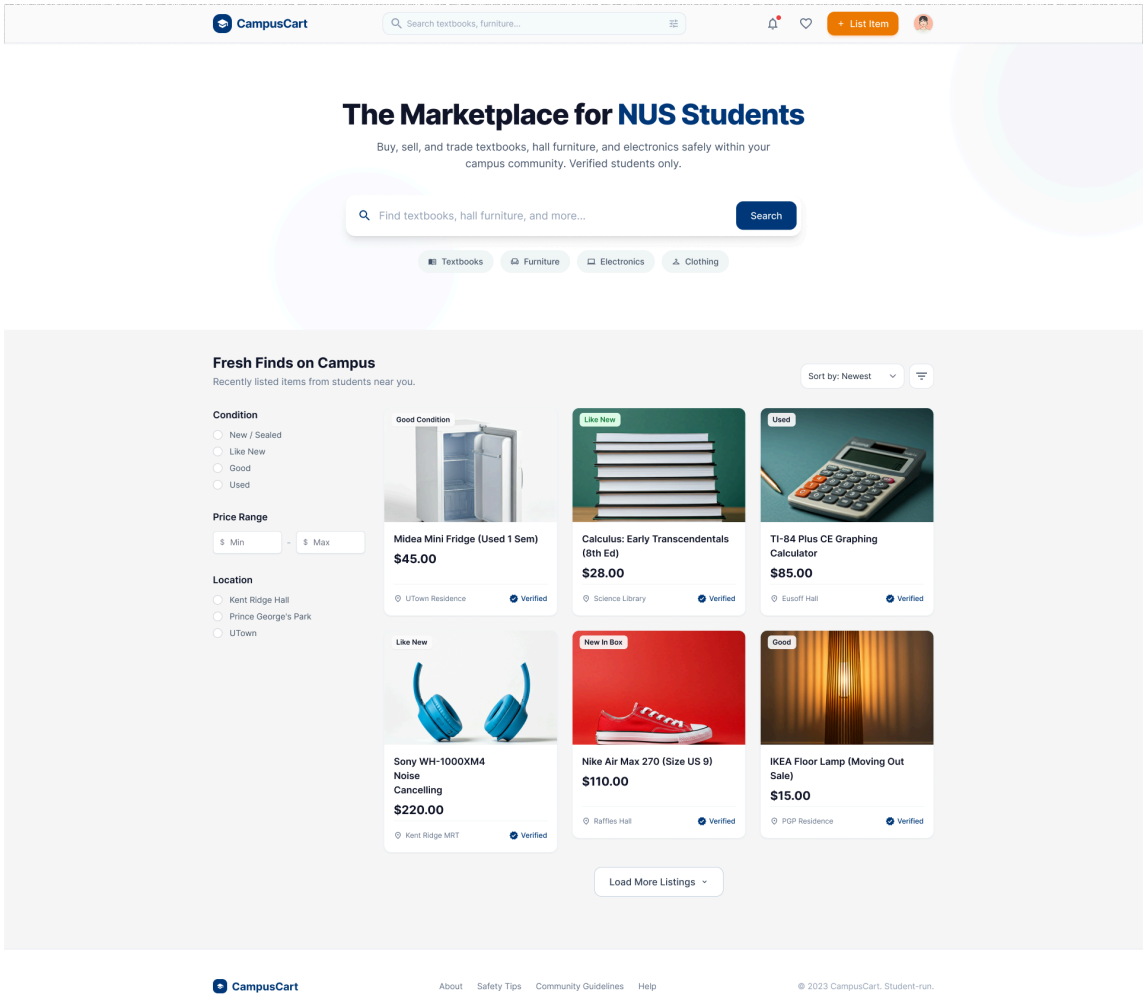



Figure 2: Marketplace homepage prototype — hero search, category filters, and listing cards with condition/location metadata.

## 2.3 Product Detail & Seller Interaction

The product detail view was designed to give buyers all the information they need to make a purchase decision without leaving the page. It features a multi-image gallery, structured item metadata (price, condition, category), a detailed text description, and a seller information card showing verification status and response time. The prominent "Make an Offer" and "Chat with Seller" buttons streamline the transaction initiation flow.

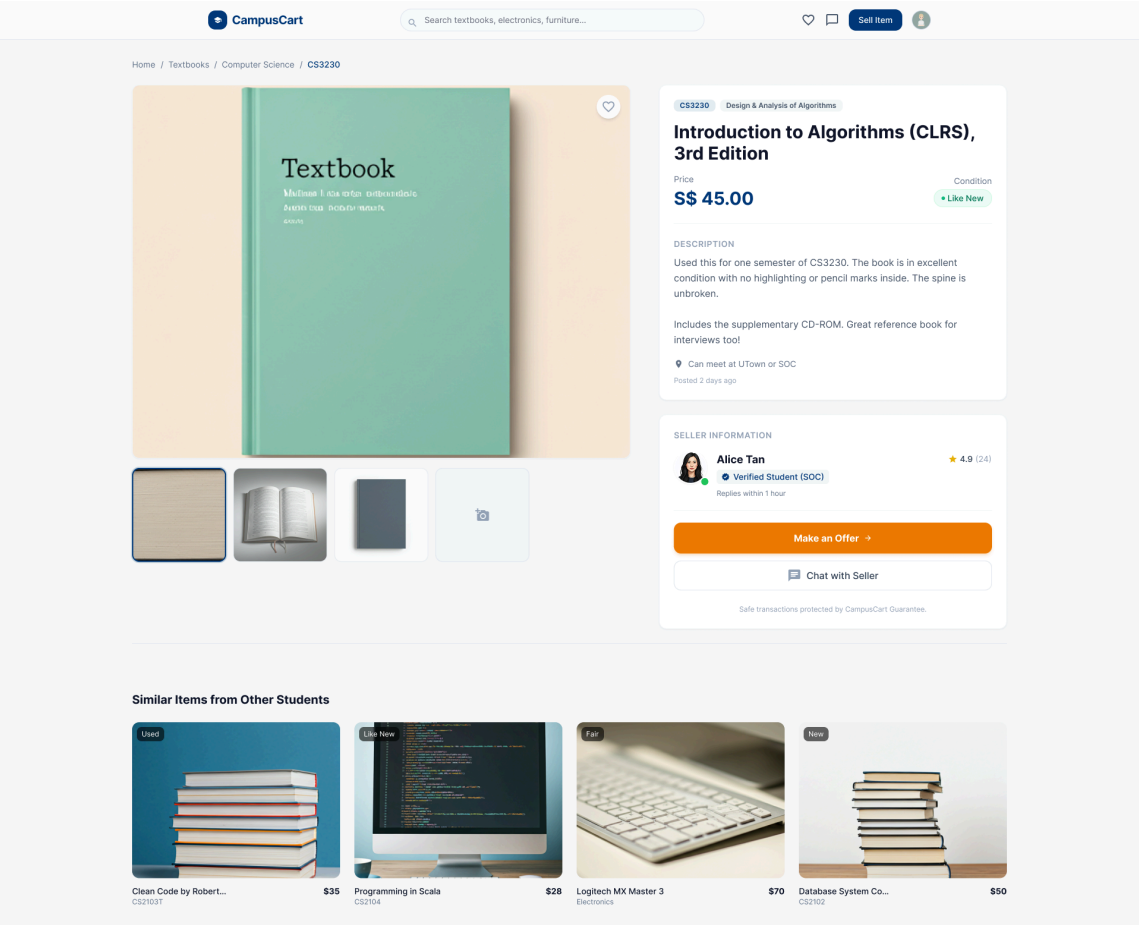
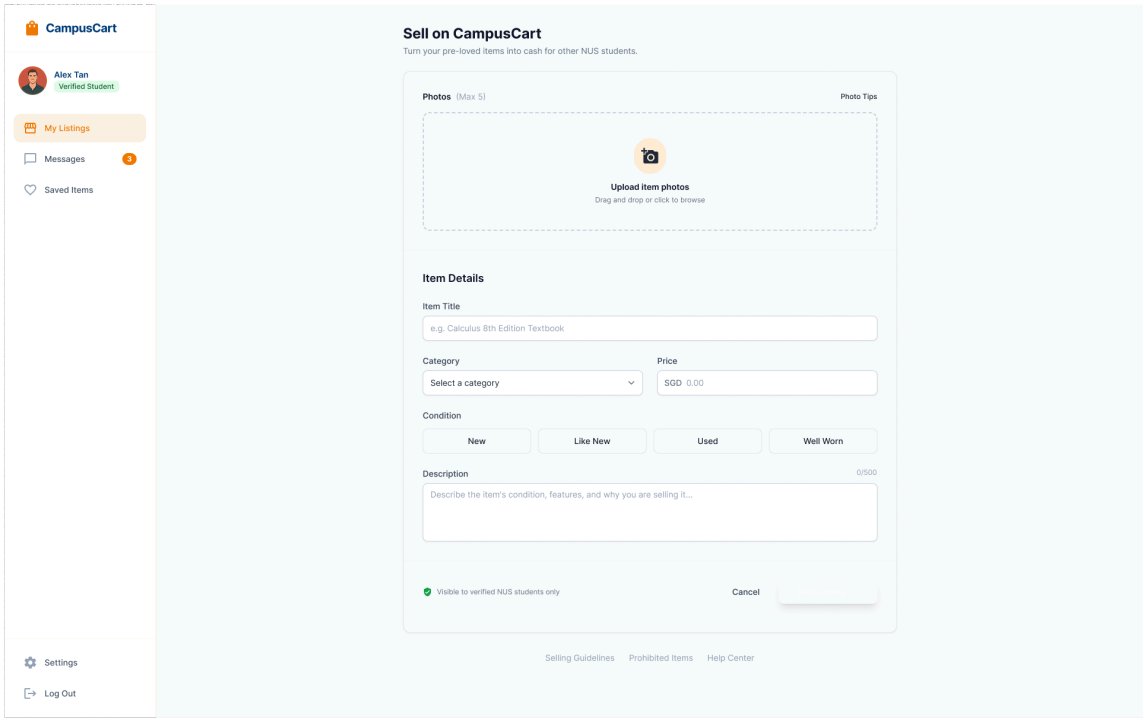



Figure 3: Product detail prototype — image gallery, item metadata, seller card, and direct offer/chat actions.

## 2.4 Seller Dashboard & Listing Management

The seller dashboard provides a comprehensive management interface with at-a-glance analytics (active listings count, total earnings, item views), tabbed navigation for active/sold/draft listings, and inline actions (edit, mark sold) for each item. This design informed the GraphQL mutation structure for listing lifecycle management.



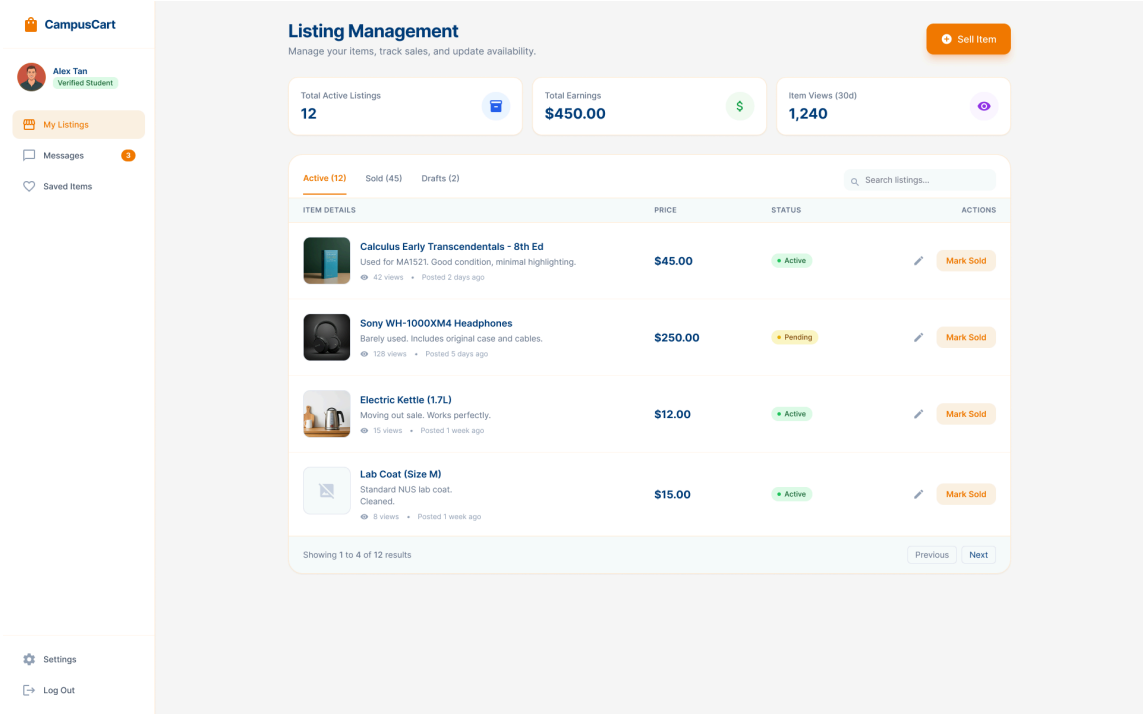
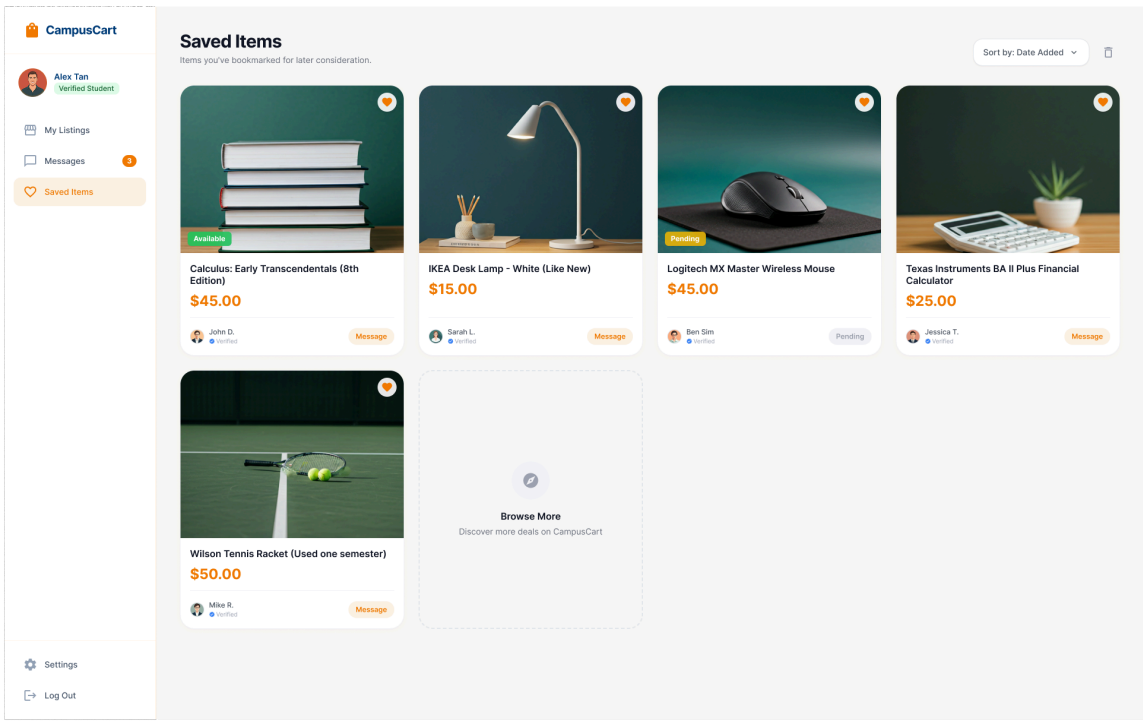




Figure 4 (left): Listing creation form with photo upload, category, condition selector, and pricing. Figure 5 (right): Seller dashboard with listing management, analytics, and status tracking.

## 2.5 Saved Items & Account Settings

The Saved Items view allows buyers to bookmark listings for later consideration, while the Account Settings page provides comprehensive profile management including personal information, contact details,

notification preferences, and account lifecycle controls.



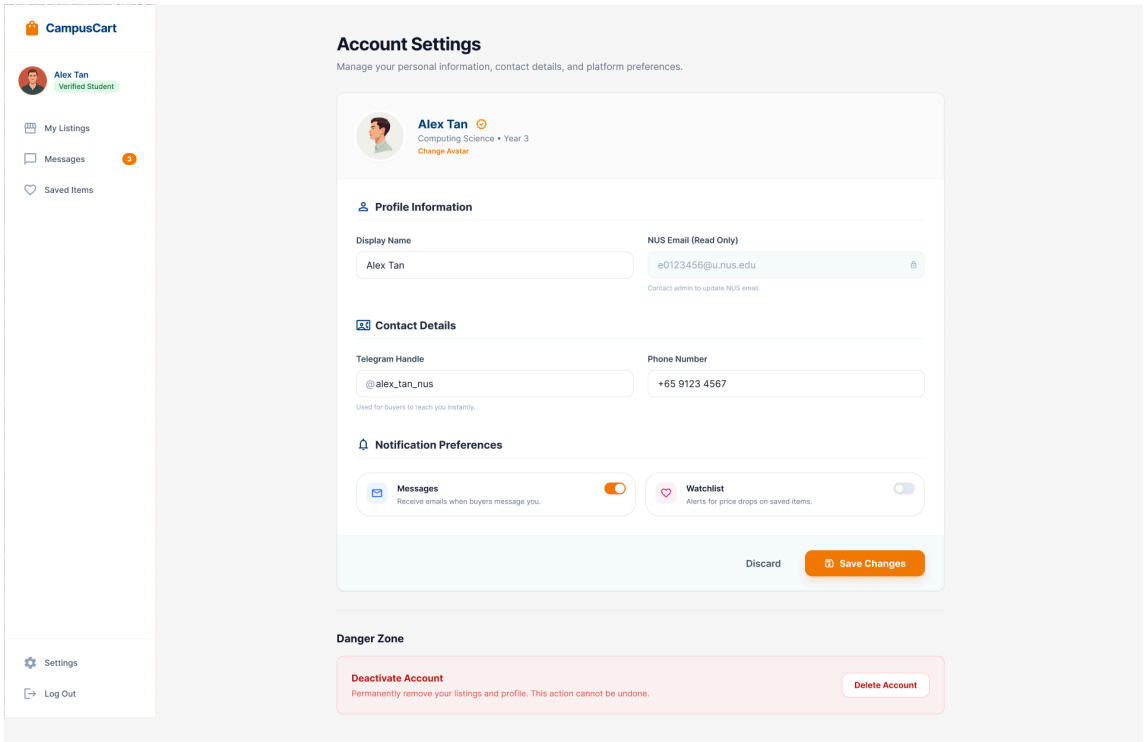




Figure 6 (left): Saved items view with bookmarked listings and availability status. Figure 7 (right): Account settings with profile information, contact details, and notification preferences.

> *These seven prototypes collectively defined the visual language, component patterns, and user flows that guided CampusCart's implementation. While the final product evolved during development —*

> *adapting to technical constraints and new insights — the Figma designs remained the north star for the team's UI/UX decisions throughout the project.*

---

## 3. Core Features & Implementation

### 3.1 Verified Authentication

To ensure trust and exclusivity, the platform utilizes a secure, session-based authentication system.

- **NUS Email Exclusivity:** Registration is strictly restricted to `@u.nus.edu` email domains. A server-side regex validator rejects all non-NUS email addresses at the point of account creation, ensuring the marketplace remains a closed, student-only community.
- **Robust Security:** The authentication layer is powered by an Express.js backend integrated with the **Better Auth** library. Passwords enforce strict security policies — minimum 8 characters, requiring uppercase letters, numbers, and special characters — with real-time validation feedback in the signup UI.
- **Protected Routing:** React Router wrappers enforce client-side session checks. Unauthenticated users are automatically redirected to the login page when attempting to access any protected route, while the backend independently validates session cookies on every GraphQL request.

### 3.2 Marketplace Architecture (Items for Sale)

- **Core Listing Lifecycle:** Sellers can create, view, edit, and delete listings. Each listing captures structured metadata including title, description, price, condition tag, category, and preferred meetup location.
- **Image Management:** Listings support photo uploads with a 5MB file-size limit. Images are persisted on the server filesystem and served via Express static middleware, decoupling binary storage from the GraphQL data layer.
- **Smart Filtering:** Buyers can browse active listings and filter by category and seller, enabling efficient discovery across the marketplace.

### 3.3 Requests Feature (Want to Buy)

In addition to selling, the platform supports a buyer-driven workflow where students can post items they are actively looking to purchase.

- **Request Lifecycle:** Students can create buying requests specifying a title, description, budget, and desired item condition.
- **Categorization & Location:** Requests are linked to the same category and location system as listings, providing a unified browse and search experience across both content types.
- **Unified Navigation:** Buying requests are easily accessible via the "+" quick-action menu and a dedicated "Want to Buy" dashboard view.

### 3.4 User Profile Enhancements

To build a personalized and trustworthy community, user profiles have been significantly expanded beyond basic authentication data.

- **Extended Profile Data:** Users can manage their bio, contact phone number, and preferred campus location.
- **Profile Customization:** Support for profile picture uploads allows users to personalize their presence on the platform.
- **Activity Tracking:** Profiles display a summary of the user's marketplace activity, including the number of active listings and open requests.

### 3.5 API Security & Authorization

All mutating API endpoints enforce server-side authentication and ownership verification.

- **Centralized Auth Helper:** A shared `requireAuth` utility is invoked across all GraphQL resolvers to ensure consistent session verification and standardized error handling.
- **GraphQL Auth Context:** Session cookies are verified on every GraphQL request via `auth.api.getSession()`, injecting the authenticated user object into the resolver context.
- **Status Validation:** Listings and requests implement strict status enums ( `active` , `sold` , `fulfilled` , `removed` ) with server-side validation to prevent illegal state transitions.
- **Cookie Security:** All frontend `fetch` calls use `credentials: 'include'` to securely transmit session cookies with every API request.

### 3.6 Data Seeding & External Product Integration

To simulate an active marketplace ecosystem and streamline developer onboarding, the platform features two complementary data-population strategies.

- **Prisma Seed Script:** A comprehensive seed script ( `prisma/seed.js` ) populates the database with demo users, categories, listings, and buying requests. Users are created through the Better Auth `signUpEmail` API, ensuring passwords are properly hashed and linked `Account` records are generated — allowing seeded users to log in immediately. The script is fully idempotent and can be re-run safely.
- **External Shopify Store Integration:** The marketplace augments its internal listings with real product data fetched from external Shopify storefronts (IUIGA, Bookshop.sg, NUS Press) via their public `/products.json` endpoints. Products are normalized into the platform's listing format with mock attributes (condition, location, verification status) assigned at runtime.

### 3.7 Client-Side Performance Caching

To avoid redundant network calls and improve page-load performance, fetched Shopify product data is cached in `sessionStorage` with a 5-minute TTL. Subsequent page visits within the cache window are served instantly from the local cache, eliminating unnecessary API calls to external storefronts.

---

## 4. Developer Experience & Collaboration Workflows

As a team of three developers working in parallel, maintaining high code quality and continuous synchronization across different local environments was critical to the project's success.

### 4.1 Code Quality Assurance

- **Husky Git Hooks:** Client-side git hooks enforced via Husky automatically check for staging errors, while custom branch-name validators prevent direct pushes to the `main` branch.
- **Linting & Formatting:** ESLint (flat config) and Prettier are configured to catch syntactic errors early and enforce a uniform, readable code style across all files.
- **CI/CD Pipeline:** GitHub Actions automatically run linting, formatting, and build-validation checks on every Pull Request, acting as a quality gate before merge.

### 4.2 Developer Synchronization & Repository Organization

- **Containerized Infrastructure:** The entire application stack (Vite frontend, Express API, and PostgreSQL database) is orchestrated using Docker Compose, providing a completely device-agnostic development environment that eliminates "works on my machine" issues.

- **Automated Database Synchronization:** Using Prisma as the ORM, schema generation is automated via `postinstall` scripts. Database states are synchronized seamlessly across all team members' machines via `pnpm prisma db push`.
- **Structured Context Documentation:** A comprehensive set of markdown documentation resides in the `resources/` directory, including developer onboarding guides, Prisma workflow references, and Git collaboration protocols. This documentation serves as the team's single source of truth, minimizing communication friction and ensuring architectural consistency.

---

# 5. Technical Architecture

## 5.1 System Architecture Overview

The diagram below illustrates the full system architecture of CampusCart — from the browser through to the data layer — and how each component is connected.

CampusCart System Architecture

## 5.2 Key Architectural Decisions

**Why GraphQL over REST?** The marketplace has multiple resource types (listings, requests, profiles) with overlapping relationships. GraphQL enables the frontend to fetch exactly the fields it needs in a single round-trip, avoiding over-fetching. The `@graphql-tools/schema` approach was chosen over Apollo Server to keep the setup lightweight — providing full GraphQL functionality without the additional abstraction layer that Apollo introduces.

**Why decouple file uploads from GraphQL?** GraphQL is not well-suited for binary data — multipart form uploads are non-standard in the spec and require special middleware. By routing image uploads through a dedicated REST endpoint backed by Multer, the GraphQL layer remains clean and type-safe while binary storage is handled separately.

**Why a shared Prisma singleton?** Node.js is single-threaded, and Prisma's connection pooling operates at the process level. Instantiating multiple `PrismaClient` instances — common in modular codebases — can exhaust the PostgreSQL connection pool. A shared singleton avoids this entirely.

**Why filesystem storage over a blob store?** For a locally-deployed academic submission with no external hosting requirement, the server filesystem is the simplest and most reliable storage option. The upload pipeline is architected so that the storage layer can be swapped (e.g., for S3 or Cloudflare R2) by modifying only the upload endpoint — the rest of the application is storage-agnostic.

## 5.3 Frontend Layer

- **Framework:** React powered by Vite for rapid compilation and Hot Module Replacement during development.
- **Styling:** Tailwind CSS v4, utilizing the modern `@theme` directive for zero-configuration, utility-first styling with customizable design tokens.

## 5.4 Backend Layer

- **API Server:** Node.js Express serving a GraphQL API via a lightweight setup ( `graphql` + `@graphql-tools/schema` ).
- **Modular GraphQL Architecture:** The backend follows a domain-driven structure. Each domain ( `listings` , `requests` , `profile` ) contains co-located schema definitions and resolvers,

aggregated by a central merge layer.

- **Schema & Input Validation:** All inputs are strictly validated (e.g., phone number formatting, budget scales, status enums) to maintain data integrity at the API boundary.
- **Shared Authentication Utility:** A centralized `requireAuth` helper provides a single entry-point for security checks across all mutations, ensuring uniform enforcement.
- **Shared Prisma Singleton:** All resolver modules share a single `PrismaClient` instance, ensuring optimal database performance and preventing connection pool exhaustion.
- **Shared Constants:** Categories and campus locations are centralized into shared constant modules, ensuring total synchronization between frontend rendering and backend validation.
- **Database:** PostgreSQL 16, provisioned via Docker, with `pgvector` extension enabled for future semantic search capabilities.
- **ORM:** Prisma v6, providing type-safe database queries natively linked to the application layer.
- **File Upload Pipeline:** Multer handles multipart form data for image uploads via dedicated REST endpoints, cleanly decoupling binary storage from the GraphQL data layer.

---

## 6. Future Implementation Roadmap

As the marketplace scales beyond its MVP, the following architectural and functional expansions are planned to enhance user discovery and platform utility:

- **AI-Powered Item Auto-Fill:** Implementing an external Vision API (e.g., Google Cloud Vision or OpenAI Vision) to identify, describe, and auto-categorize item listings from a user's photo upload. The image upload pipeline (Multer REST endpoint) and listing creation API are already in place — the AI call would slot in as an intermediate processing step before the listing is saved.
- **Semantic Search via pgvector:** The PostgreSQL database already has the `pgvector` extension enabled. The natural next step is to generate vector embeddings for listing titles and descriptions (using a lightweight embedding model) and store them alongside listing records. This would enable similarity-based search — surfacing semantically related listings even when keyword terms don't match exactly.
- **Omni-Channel & Mobile Integration:** Enhancing the frontend as a mobile-first progressive web app experience to leverage native device capabilities, such as immediate camera access for photo uploads.
- **Authentication Expansions:** Supplementing the current email validation framework with scalable OAuth 2.0 pipelines (e.g., Sign in with Google or GitHub). Better Auth already supports OAuth providers — this would be a configuration-level addition rather than an architectural change.